

Exercice 6 — Gestion des transactions & rollback

EILMS — Java Enterprise (Step 1 & Step 2)

Énoncé

Que se passe-t-il si une `RuntimeException` est levée entre `em.persist(item)` et `em.getTransaction().commit()`, sans `try/catch` ?

```
em.getTransaction().begin();
WarehouseItemEntity item = new WarehouseItemEntity("ITEM-201", "Router", 25);
em.persist(item);
// ... une RuntimeException est levée ici ...
em.getTransaction().commit(); // jamais atteint
```

Correction

Le `commit()` n'est jamais exécuté, donc **rien n'est écrit en base de données** — la transaction reste ouverte et non validée. L'exception remonte ensuite la pile d'appels jusqu'à être catchée plus haut, ou termine le programme si elle ne l'est jamais.

Bonne pratique — englober dans un `try/catch/finally` :

```
EntityTransaction tx = em.getTransaction();
try {
    tx.begin();
    WarehouseItemEntity item = new WarehouseItemEntity("ITEM-201", "Router", 25);
    em.persist(item);
    tx.commit();
} catch (RuntimeException e) {
    if (tx.isActive()) {
        tx.rollback();
    }
    throw e;
} finally {
    em.close();
}
```

`rollback()` annule explicitement toute modification en attente dans le contexte de persistance, garantissant qu'aucun état partiel/incohérent ne soit écrit en base. C'est le « A » (Atomicité) des propriétés ACID : soit toute la transaction réussit, soit aucune partie n'est appliquée.